OpenGL编程实践

徐倩

Email: xuqian@graphics.pku.edu.cn

Agenda

- OpenGL简介
- OpenGL编程模板
  - 窗口管理
  - 视图
  - 清空操作
  - 双缓冲
  - 回调函数
- 绘制实例
  - 画条线、画个面
  - 画个立方体，并让它动起来
- 使用 Windows API设置OpenGL环境
- 编程常见问题

OpenGL简介

OpenGL是什么？

OpenGL全称：Open Graphics Library，是专业的开放图形库。

编译环境：

OpenGL在任何常用编译环境都没问题，比如，大家最常用的visual studio

OpenGL库：

常用函数库组合：GL+GLU+GLUT(适合入门阶段) 或 GL+GLU+WGL(windows相关的绘制程序)

函数库的使用方式（以Visual Studio 为例）：

将“*.h”文件添加到以下路径：Visual Studio安装路径/VC/include，程序中声明：#include <*.h>

将“*.lib”文件添加到以下路径：Visual Studio安装路径/VC/lib

将“*.dll”文件添加到以下路径：Visual Studio安装路径/VC/bin 或者 C:/Windows/System32

OpenGL编程模板

```
#include <GL/glut.h>
#include <stdio.h>

void init(void)
{
   /* 选择缓冲区的颜色*/
   glClearColor (0.0, 0.0, 0.0, 0.0);

    /*打开深度测试*/
    glEnable( GL_DEPTH_TEST );

    /* 设置投影参数*/
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluPerspective( 90, 1, 0.1, 10.0 );
    glMatrixMode( GL_MODELVIEW);
    gluLookAt(0.0,0.0,0.0,0.0,0.0,-1.0,0.0,1.0,0.0);
}
 void display(void)
{
  /* 清除缓冲区  */
   glClear (GL_COLOR_BUFFER_BIT |
GL_DEPTH_BUFFER_BIT);

  /* 需要画什么，将绘制代码添加到这里 */

/* 交换一个缓冲区 */
glutSwapBuffers();
}
```

视图

清除缓冲区

双缓冲

窗口管理

注册回调函数

```
int main(int argc, char** argv)
{
  /* 初始化窗口 */
   glutInit(&argc, argv);
   glutInitDisplayMode
   (GLUT_DOUBLE | GLUT_RGBA |
       GLUT_DEPTH);
   glutInitWindowSize (250, 250);
   glutInitWindowPosition (100, 100);
   glutCreateWindow ("Hello World!");

  /* 初始化设置部分参数 */
   init ();

   /* 注册显示回调函数*/
   glutDisplayFunc(display);

  /* 启动程序。事件处理循环开始启动 */
   glutMainLoop();

   return 0;
}
```

OpenGL编程模板

一个简单基本的OpenGL程序需要包含的元素有：

- **窗口管理**：初始化一个OpenGL绘制窗口和绘制环境
- **视图设置**：设定相机位置、方向以及裁剪方式
- **注册回调函数**：注册要回调的函数，通常情况下，至少需要注册显示回调函数(glutDisplayFunc(display))
- **清除缓冲区**：每一帧绘制开始时，需要清除缓冲区
- **交换缓冲区**：如果使用了双缓冲，在每帧绘制结束后，需要交换缓冲区

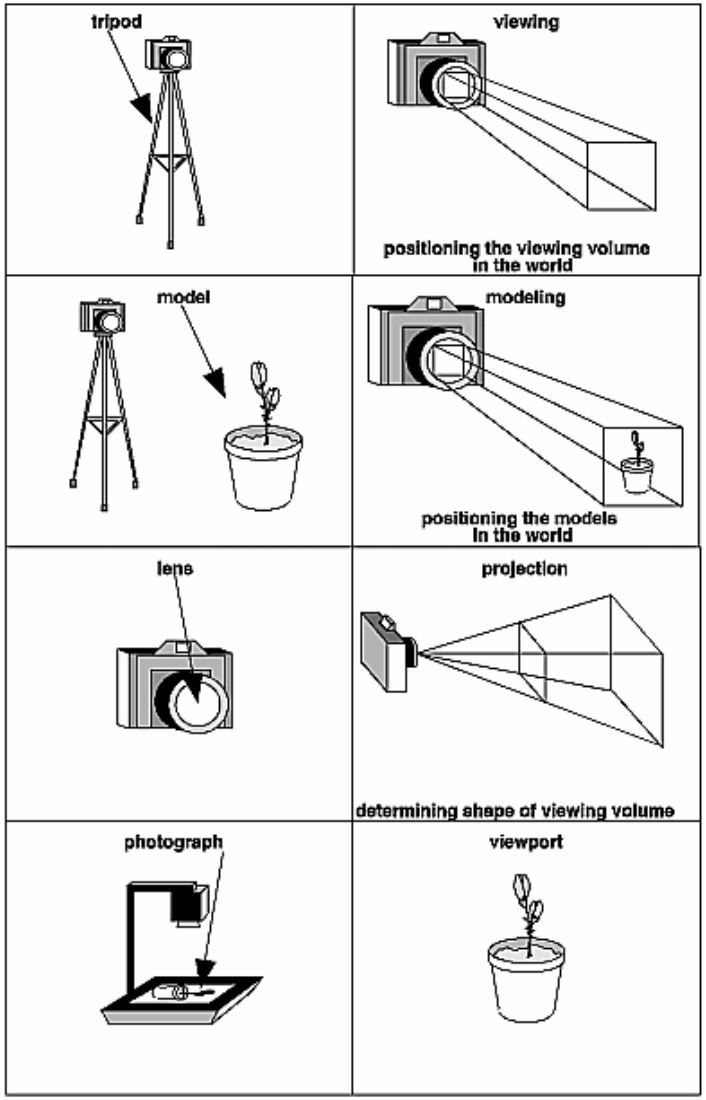
窗口管理

glut库的窗口管理

```
//初始化glut
 glutInit(&argc, argv);
//设置窗口的绘制模式
 glutInitDisplayMode (GLUT_DOUBLE | GLUT_RGBA | GLUT_DEPTH);
//窗口的大小
glutInitWindowSize (250, 250);
//窗口位置
glutInitWindowPosition (100, 100);
//生成窗口
glutCreateWindow ("Hello World!");
```

视图

照相机类比法



- **视图变换：**将照相机固定在三脚架
  例：gluLookAt
- **模型变换：**使场景处于合适的位置
  例：glTranslatef
- **投影变换：**调节镜头或调整放大倍数
  例： gluPerspective
- **视区变换：**决定最终相片的尺寸大小
  例：glViewPort

视图

视图和模型变换

视图变换和模型变换实际效果是一致的，都相当于对空间中的一点乘以一个矩阵，使得它的坐标发生一定的改变。

$$\begin{bmatrix} x \\ y \\ z \\ w \end{bmatrix}$$

object coordinates

$$M = \begin{bmatrix} m_1 & m_5 & m_9 & m_{13} \\ m_2 & m_6 & m_{10} & m_{14} \\ m_3 & m_7 & m_{11} & m_{15} \\ m_4 & m_8 & m_{12} & m_{16} \end{bmatrix}$$

视图

视图变换

gluLookAt( ) 函数：

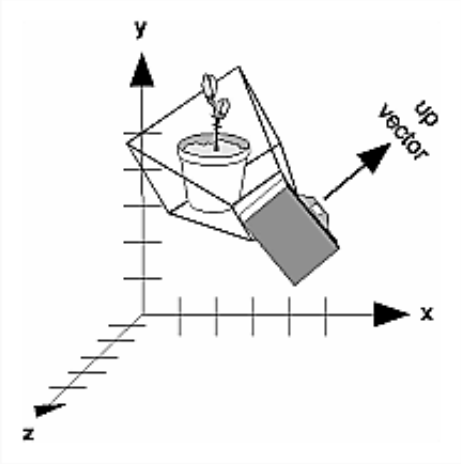
void gluLookAt(GLdouble eyex,GLdouble eyey,GLdouble eyez, Gldouble centerx, GLdouble centery, GLdouble centerz, GLdouble upx, GLdouble upy, GLdouble upz)；

说明：

(eyex, eyey, eyez)是观察点坐标，(centerx, centery, centerz)是视线上任意一点的坐标，(upx, upy, upz)表明哪个方向是朝上的。

默认情况下，(eyex, eyey, eyez)=(0.0,0.0,0.0),

(centerx, centery, centerz)=(0.0,0.0,-100.0), (upx, upy, upz)=(0.0,1.0,0.0);



gluLookAt(4.0, 2.0, 1.0, 2.0, 4.0, -3.0, 2.0, 2.0, -1.0);

视图

模型变换

三种模型变换：

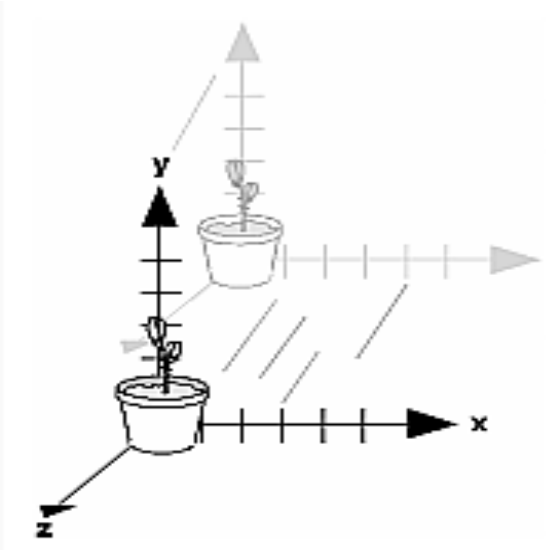
- 平移变换
- 旋转变换
- 缩放变换

视图

平移变换

平移变换：glTranslate { fd }(x, y, z);

x, y, z分别表示对象在XYZ轴上的平移距离。

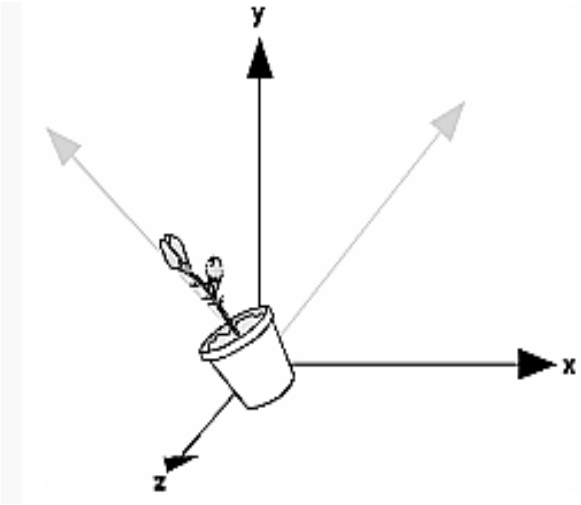


glTranslatef(0.0,0.0,-5.0);

旋转变换

旋转变换：glRotate { fd }(angle, x, y, z);

以逆时针方向绕着从原点到点(x, y, z)的直线进行旋转，angle指定了旋转的角度

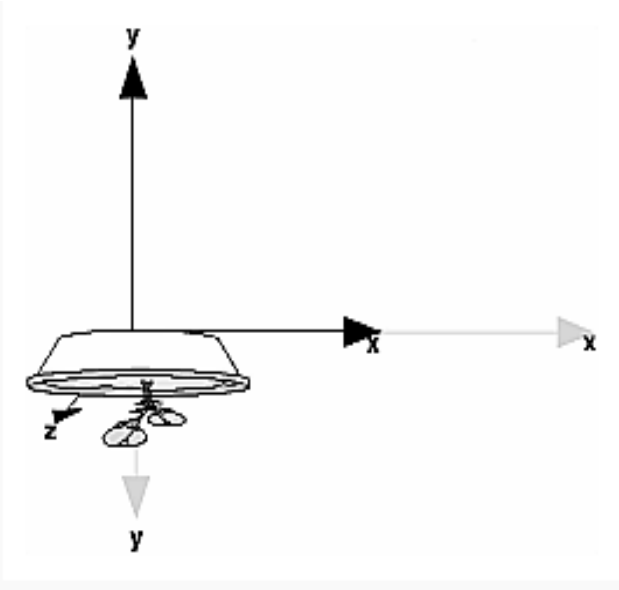


glRotatef(45, 0.0, 0.0, 1.0);

缩放变换

缩放变换：glScale { fd } (x, y, z);

x, y, z分别表示物体在XYZ轴上的缩放倍数，可以为负数
glScalef是唯一可以改变物体大小的变换



glScalef(2.0, -0.5, 1.0);

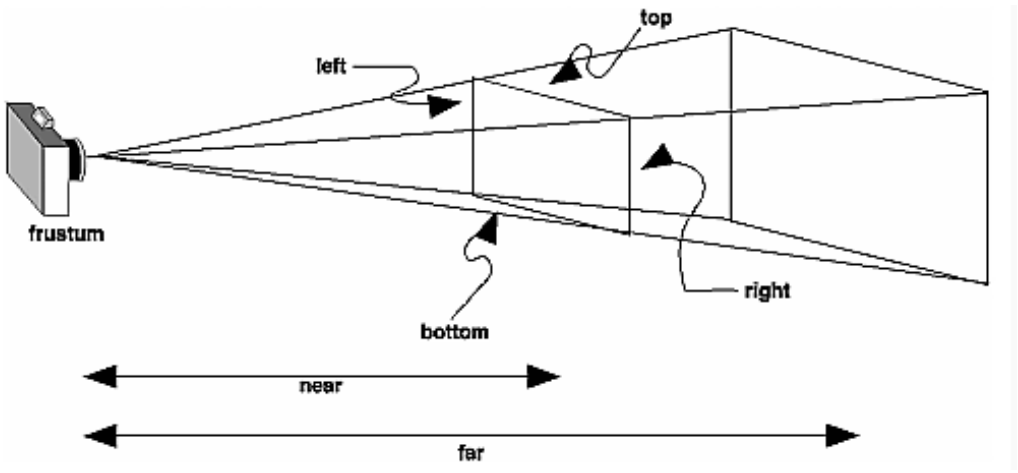
视图

投影变换

透视投影：定义了一个视景体，该视景体可以看做一个平截头体，用于裁剪和投影
位于视景体之内的物体将被投影到相机的位置，有近大远小的效果

glFrustum(GLdouble left, GLdouble right, GLdouble bottom, GLdouble top, GL double near, GLdouble far);

说明：(left, bottom, -near) 和 (right, top,-near）分指定了近裁剪平面的左上角和右上角的坐标。near和far分别表示从观察点到近侧和远侧裁剪平面的距离。



视图

投影变换

透视投影：

gluPerspective(GLdouble fogy, GLdouble aspect, GLdouble near，GLdouble far)；

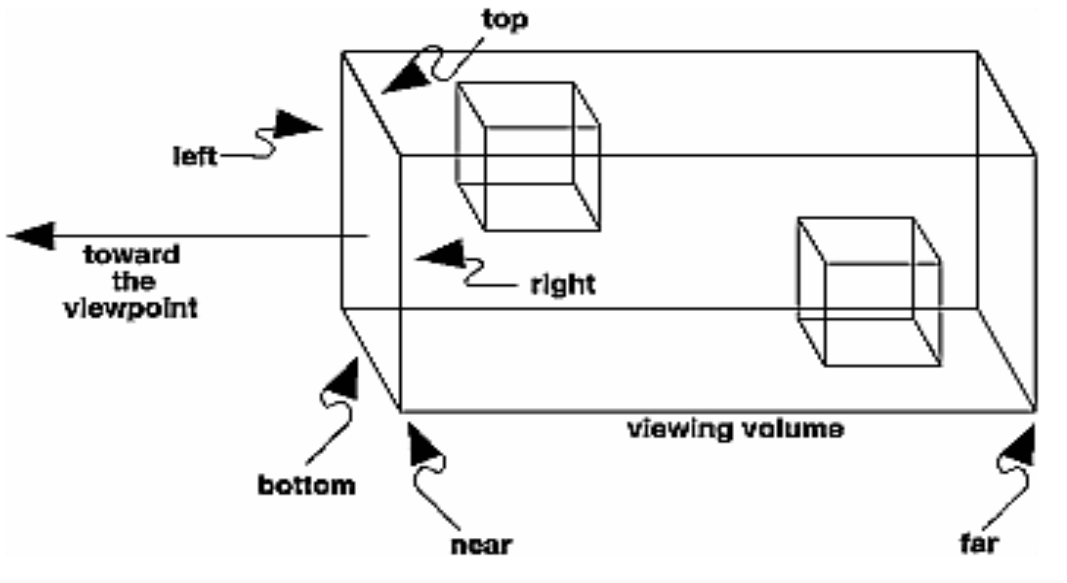
说明：创建了一个对称的视景体，fovy是yz平面上的视野角度，范围在[0.0,180.0]，aspect是平截头的纵横比

视图

投影变换

正交投影：视景体是平行的长方体，物体经过投影后，保持实际大小，不会有近大远小的效果

glOrtho(GLdouble left, GLdouble right, GLdouble bottom， GLdouble top， GLdouble near，GLdouble far)；

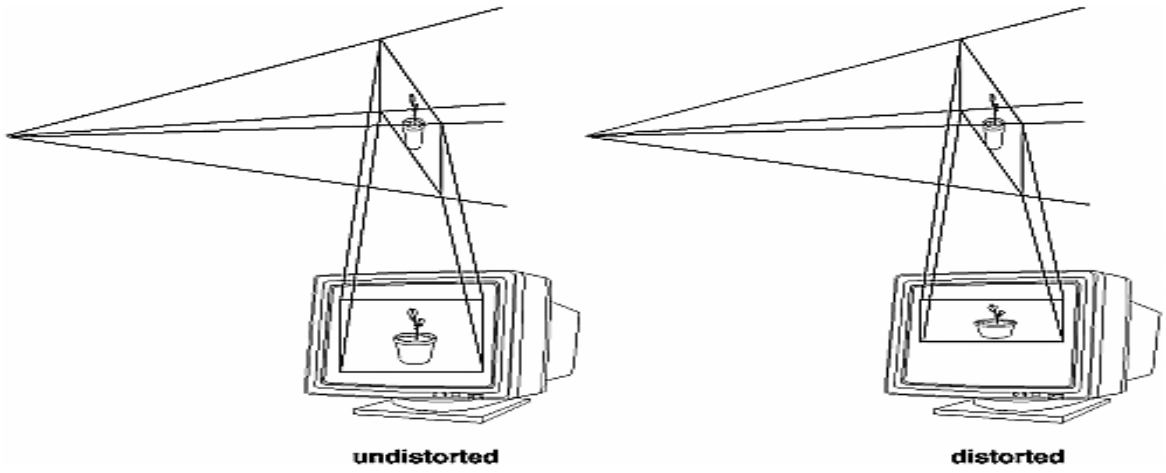


视图

视区变换

void glViewPort(GLint x, GLint y, GLint width, GLint height);

说明：视区变换对应于选择被冲洗照片的大小这个阶段，(x, y)指定了视口的左下角坐标，width和height指定了窗口大小。



视图

矩阵堆栈的操作

- 投影变换、模型变换、视图变换实质都是矩阵操作：
  - 投影变换会影响到**投影矩阵**
  - 模型变换和视图变换会影响到**模型视图矩阵**
- OpenGL提供的可以操作的矩阵堆栈有两类：**投影矩阵堆栈和模型视图矩阵堆栈**。
  - 投影矩阵堆栈：可以存储2个4 X 4矩阵，但是，一般不需要进行投影矩阵的组合
  - 模型视图矩阵堆栈：至少可以包括32个4 X 4的矩阵；
- 对矩阵堆栈的**操作**:
  - glPushMatrix( ) ：复制一份当前矩阵，将拷贝添加到堆栈顶部。相当于保存状态
  - glPopMatrix( )：将堆栈顶部的矩阵弹出，并且销毁弹出的内容。相当于恢复状态

视图

矩阵堆栈的操作

利用glPushMatrix和glPopMatrix完成一个简单应用：画一辆车，包含一个车身、两个轮子和多个螺丝

```
draw_wheel_and_bolts( )    //画一个轮子以及多个螺丝
 {
        long i;
        draw_wheel();   //画一个轮子

        for(int  i=0;i<5;i++) {    //在5个不同的位置画螺丝

            glPushMatrix();                    //保存矩阵状态
            glRotate(72.0*i,0.0,0.0,1.0);      //设置好螺丝的位置
            glTranslatef(3.0,0.0,0.0);
            draw_bolt();                       //画螺丝
            glPopMatrix();                     //恢复矩阵状态
        }
  };


  draw_car ( )   //画车子
  {
        draw_car_body();   //画车身

        glPushMatrix();
        glTranslate(40,0,30);
        draw_wheel_and_bolts();   //画一个轮子和多个螺丝
        glPopMatrix();

         glPushMatrix();
         glTranslate(40,0,-30);
         draw_wheel_and_bolts();  //换个位置画轮子和多个螺丝
         glPopMatrix();
  };
```

清空操作

- 清除背景

  void glClear(GLbitfield mask)
  GL_COLOR_BUFFER_BIT：颜色缓存
  GL_DEPTH_BUFFER_BIT：深度缓存

- 先设置清空值，再执行清空操作

  void glClearColor(GLclampf red,GLclampf green,GLclampf blue,GLclampf alpha);
  void glClearDepth(GLclampd depth);

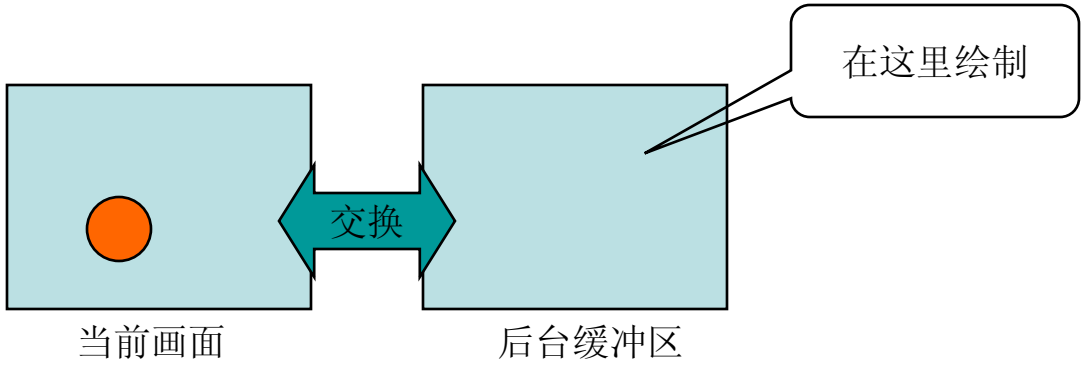
- 使用“|”运算符同时清除多项

  glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)

双缓冲

初始化：glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGB); //设置使用双缓冲

绘制完一帧的末尾：glutSwapBuffers() //交换前后缓冲区



双缓冲实现动画：前台缓存显示当前帧，后台缓存同时进行后续帧的场景计算，生成画面

回调函数

- 回调函数由用户提供，但由系统（这里是GLUT）调用
- 用户按照系统给定的格式提供回调函数
- 系统通过注册函数给定回调函数的格式
- 注册函数和回调函数一一对应，注册函数负责在系统中登记回调函数

回调函数详细内容见 第三讲《OpenGL图形系统概述》讲义的p70-82

画条线，画个面

```
void display(void)
{
    /* 清除缓冲区*/
    glClear (GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    /* 画条线*/
    glBegin( GL_LINES );
    glColor3f( 0.0, 1.0, 0.0);
    glVertex3f( 1.0, 0.0, -1.0 );  //顶点1
    glVertex3f( 0.5, 1.0, -1.0);  //顶点2
    glEnd();

    /* 画个面*/
    glBegin( GL_QUADS );
    glColor3f( 1.0, 0.0, 0.0 );
    glVertex3f( 0.0, 0.0, -1.0 );  //顶点1
    glColor3f( 0.0,0.5,0.5 );
    glVertex3f( 0.5, 0.0, -1.0 );  //顶点2
    glColor3f( 1.0, 0.0, 0.0 );
    glVertex3f( 0.5, 0.5, -1.0 );  //顶点3
    glColor3f( 0.8, 0.8, 0.8 );
    glVertex3f( 0.0, 0.5, -1.0 );  //顶点4
    glEnd ( );

    /* 交换一个缓冲区*/
    glutSwapBuffers();
}
```

画个立方体，并让它动起来

```
#include <GL/glut.h>
#include <stdio.h>

int angle = 0;  //旋转角度

void init(void)
{
     /* 选择缓冲区的颜色*/
    glClearColor (0.0, 0.0, 0.0, 0.0);

     /*打开深度测试*/
    glEnable( GL_DEPTH_TEST );

    /* 设置投影参数*/
   glMatrixMode(GL_PROJECTION);
   glLoadIdentity();
   gluPerspective( 90, 1, 0.1, 10.0 );
   glMatrixMode( GL_MODELVIEW);
   gluLookAt(0.5,0.5,0.0,0.5,0.5,-1.0,0.0,1.0,0.0);
}

/*键盘控制*/
void keyboard(unsigned char key, int x, int y)
{
    switch (key)
    {
       case 'r':
              angle = (angle+5)%360;
              glutPostRedisplay();   //重新调用display函数
              break;
        case 'R':
              angle = (angle-5)%360;
              glutPostRedisplay();   //重新调用display函数
              break;
         case 27:
              exit(0);
              break;
    }
}
```

```
void display(void)
{
        /* 清除缓冲区*/
        glClear (GL_COLOR_BUFFER_BIT |
GL_DEPTH_BUFFER_BIT);

        /* 画个长方体*/
        glPushMatrix();          //保存矩阵状态

        glRotated( angle, 0.5, 0.5, -1.5 );   //物体旋转一
定角度

         glColor3f( 1.0, 0.0, 0.0 );
         glBegin( GL_QUADS );
         glVertex3f( 0.0, 0.0, -1.0 ); //顶点
         glVertex3f( 1.0, 0.0, -1.0 );
         glVertex3f( 1.0, 1.0, -1.0 );
         glVertex3f( 0.0, 1.0, -1.0 );
         glEnd ( );

         glColor3f( 0.0, 1.0, 0.0 );
         glBegin( GL_QUADS );
         glVertex3f( 0.0, 0.0, -2.0 ); //顶点
         glVertex3f( 0.0, 1.0, -2.0 );
         glVertex3f( 1.0, 1.0, -2.0 );
         glVertex3f( 1.0, 0.0, -2.0 );
         glEnd ( );

         glColor3f( 0.0, 0.0, 1.0 );
         glBegin( GL_QUADS );
         glVertex3f( 0.0, 0.0, -1.0 ); //顶点
         glVertex3f( 0.0, 0.0, -2.0 );
         glVertex3f( 1.0, 0.0, -2.0 );
         glVertex3f( 1.0, 0.0, -1.0 );
         glEnd ( );
```

画个立方体，并让它动起来

```
glColor3f( 1.0, 1.0, 0.0 );
glBegin( GL_QUADS );
glVertex3f( 0.0, 1.0, -1.0 ); //顶点
glVertex3f( 1.0, 1.0, -1.0 );
glVertex3f( 1.0, 1.0, -2.0 );
glVertex3f( 0.0, 1.0, -2.0 );  //顶点
glEnd ( );

glColor3f( 0.0, 1.0, 1.0 );
glBegin( GL_QUADS );
glVertex3f( 0.0, 0.0, -2.0 ); //顶点
glVertex3f( 0.0, 0.0, -1.0 );
glVertex3f( 0.0, 1.0, -1.0 );
glVertex3f( 0.0, 1.0, -2.0 );
glEnd ( );

glColor3f( 1.0, 0.0, 1.0 );
glBegin( GL_QUADS );
glVertex3f( 1.0, 0.0, -2.0 ); //顶点
glVertex3f( 1.0, 1.0, -2.0);
glVertex3f( 1.0, 1.0, -1.0 );
glVertex3f( 1.0, 0.0, -1.0 );
glEnd ( );

glPopMatrix();   //恢复矩阵状态

/* 交换一个缓冲区*/
glutSwapBuffers();

}
```

```
int main(int argc, char** argv)
{
    printf( "------ 按'r'，长方体逆时针旋转\n");
    printf( "------ 按'R'，长方体顺时针旋转\n");
    printf( "------ 按ESC，退出系统\n");

    /* 初始化窗口*/
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_DOUBLE | GLUT_RGBA | GLUT_DEPTH);
    glutInitWindowSize (250, 250);
    glutInitWindowPosition (100, 100);
    glutCreateWindow ("Hello World!");


    /* 初始化设置部分参数*/
    init ();

    /* 注册显示回调函数*/
    glutDisplayFunc(display);

    /* 注册键盘回调函数*/
    glutKeyboardFunc(keyboard);

    /* 启动程序。事件处理循环开始启动*/
    glutMainLoop();

    return 0;
}
```

Windows API 建立 OpenGL环境

- 在Windows中，OpenGL命令是通过**OpenGL Render Context**（绘制环境，以下简称RC）来执行的。这个RC是OpenGL和Windows之间的纽带。

- 每个RC创建时，**必须指定一个DC**（Windows Device Context 也就是那个通常用于GDI的设备环境DC）。RC的绘制目标，就是这个DC所对应的窗口。每次RC创建时，应该**设置一下DC的像素格式**。

- 同一个时刻，一个窗口可能对应多个RC，但**一个线程只能有一个被激活的RC。每个线程有且只能拥有一个RC**。

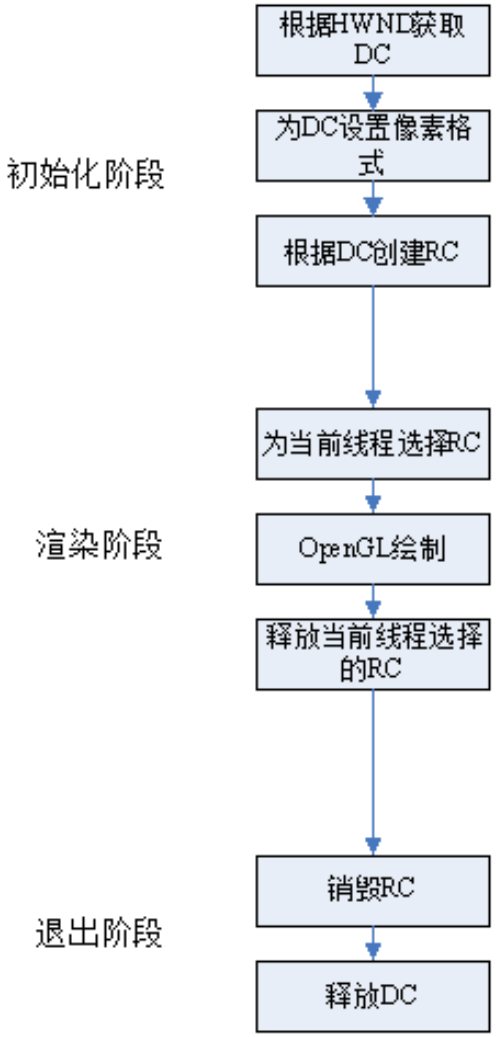
Windows API 建立 OpenGL环境

Windows提供了一组用于绘制环境（RC）管理的函数：

- **wglCreateContext：**创建一个RC。
- **wglMakeCurrent：**为线程指定当前RC。
- **wglGetCurrentContext：**获取一个线程的当前RC。
- **wglGetCurrentDC：**获取当前线程RC所对应的DC。
- **wglDeleteContext：**删除一个RC。

Windows API 建立 OpenGL环境

使用Windows API 创建OpenGL环境流程如下：



Windows API 建立 OpenGL环境

初始化

```
HDC   g_HDC;   //windows设备环境句柄
HGLRC hrc;     //OpenGL渲染上下文环境句柄

g_HDC=GetDC(hwnd);  //得到当前窗口的DC （设备环境）

int   nPixelFormat;
static   PIXELFORMATDESCRIPTOR   pfd= {
sizeof ( PIXELFORMATDESCRIPTOR ),  //数据结构大小
1,  //版本号,总设为1
PFD_DRAW_TO_WINDOW,  //支持窗口
PFD_SUPPORT_OPENGL,  //支持OpenGl
PFD_DOUBLEBUFFER, //支持双缓存
PFD_TYPE_RGBA,  //RGBA颜色模式
32,  //32bit颜色模式
0,0,0,0,0,0,  //忽略颜色位,不使用
0,  //无alpha缓存
0,  //忽略偏移位
0, //无累计缓存
0,0,0,0, //忽略累积位
16,  // 16bit z-buffer(z缓存)大小
0, //无模板缓存
0, //无辅助缓存
PFD_MAIN_PLANE, //主绘制平面
0, //保留的数据项
0,0,0}; //忽略层面研掩膜
```

```
//选择最匹配的像素格式,返回索引值
nPixelFormat=ChoosePixelFormat(g_HDC,&pfd);

//设置设备环境的像素格式
SetPixelFormat(g_HDC,nPixelFormat,&pfd);

//创建RC（绘制环境）
hrc = wglCreateContext(hdc);
```

Windows API 建立 OpenGL环境

绘制

```
//为当前线程确定好绘制设备和环境
wglMakeCurrent(hdc,hrc);

//调用OpenGL接口进行绘制


//交换缓冲区
SwapBuffers(m_hDC);

//取消当前线程选中的RC
wglMakeCurrent( NULL,NULL );
```

编程常见问题

为啥窗口里啥都没画出来呢？

- 矩阵设置有问题：裁剪窗口将物体裁掉了、观察方向不对、视点位置不对
- 双缓冲模式，却没有在每一帧的末尾，交换缓冲区
- 忘记在每一帧的开始要清除缓冲区
- 颜色和屏幕底色相同了
- glPushMatrix 和 glPopMatrix没有对应使用

物体移动时，不停的闪烁

- 没有使用双缓冲
- 编程环境初始化出问题了
- 设置投影矩阵时，znear的值过小，导致zfar/znear过大

编程常见问题

几个可以用来查错的函数：

- 错误类型查询：

```
GLenum glGetError( void );
```

- 开关状态查询：

```
GLboolean glIsEnabled(GLenum cap);
```

- 变量状态查询：

```
void glGetBooleanv( GLenum pname, GLboolean *params );
void glGetDoublev( GLenum pname, GLdouble *params );
void glGetFloatv( GLenum pname, GLfloat * params );
void glGetIntegerv( GLenum pname, GLint * params );
```

Enjoy OpenGL~

Follow me on weibo.sina:
http://weibo.com/circlepig